

Graded Where-Clauses

Non-crisp truth values as the syntax of non-determinism resolution
in the rho calculus, with quantum and probabilistic-logic instances

L. G. Meredith
F1R3FLY.io

F1R3FLY.io working note — August 2026
Version 4: dilation, heralds, and the price of erasure

Abstract

The rho calculus is silent about how the non-determinism of a race is resolved. An implementation must supply that mechanism, and different mechanisms give different executions of the same program: RSpace supplies the physical races of cores on a CPU, and one may equally supply the Born rule. We take this seriously as a design principle and ask what syntax a programmer needs in order to *arrange* a resolution mechanism rather than merely inherit one.

We argue, first, that no repair of full abstraction over rho contexts can expose the additional structure a quantum resolver supplies, because no rho context says anything about how non-determinism is resolved; the mismatch is not in the class of contexts but in the type of observation. We then show that unmodified rho, read as a sum over its possible futures, already interferes — but only on structurally indistinguishable racers, and only constructively, because the message left behind by a race is a which-path record.

The design that follows is a single change: the **where** clause of the for-comprehension is valued in a commutative semiring $\mathbb{V}$ rather than in the Booleans, and is evaluated *per candidate match* rather than per communication. Everything else is unchanged. We show that (i) weight maps in the sense of the weighted-GSLT programme are exactly the linear normal forms of graded clauses, so the partition discipline and the update-function machinery both disappear; (ii) the join operator $\&$ is already a recombiner, so the which-path erasure that interference requires needs no new term former; and (iii) the graded clause on an m -fold join over m data is a scattering matrix, with the amplitude of the symmetric outcome given by its permanent. Boolean $\mathbb{V}$ recovers rholang 1.4 exactly; $\mathbb{R}_{\geq 0}$ gives stochastic execution; $\mathbb{C}$ gives interference, including cancellation of an operationally reachable outcome.

As a worked example we build Shor’s algorithm in the fragment, with every number computed by a simulator that enumerates candidates rather than multiplying gate matrices. The whole program is given — the three gadget contracts, modular exponentiation, and the instance at $N = 15$ — emitted from the same gate list the simulator executes, so the code and the numbers cannot drift apart. Universality is easy and modular exponentiation is ordinary rholang, but the exercise turns up a hazard rather than a triumph: with clauses unconstrained, renormalising at observation is post-selection, and the fragment computes PP.

This version contains the hazard by construction rather than by side condition. Every receipt is dilated at the site — not at a global step, which interleaving makes ill-defined — and the dilation’s ancilla is not traced, nor projected, but *addressed*: it is an ordinary message on a channel named by quoting the site, carrying one bit that says whether the step preserved norm. We show that this bit is the only thing the message may carry, since a herald naming the candidate would destroy interference at every site, and that heralding is therefore inert exactly on the fragment where isometry is decidable. Under the amendment no step loses norm, so post-selection ceases to be an operation of the semantics and becomes a retry loop the programmer writes and the resource ledger charges: the filter that was free

costs 2^n firings. We also record why we do not take the obvious alternative — restricting to the linear fragment, where every bound name is used exactly once — namely that the circuit encoding’s expressiveness is bought with weakening, and a weakening-free step is block-diagonal, hence an isometry and not a gate.

Finally we observe that the same slot accepts truth values in the sense of Goertzel’s Probabilistic Logic Networks. Valuing where-clauses by PLN inference makes MeTTaIL’s execution engine and its reasoning engine the same loop: what fires next is decided by what the system currently believes. We give the resolution algebra this requires, and are explicit about which part of PLN’s truth-value arithmetic supplies values and which part would have to be weakened to supply the algebra. A second worked example — a mortal forager whose hypothesis is a PLN truth value in a **where** guard — shows what the grading buys: a Boolean guard has no coordinate in which to record disappointment and cannot learn to stop, exploration and exploitation cease to be scheduled, and PLN’s personality parameter, elsewhere set by taste, acquires an interior optimum fixed by the risk of the environment.

Contents

1	Introduction	3
1.1	Where the choice is	3
1.2	The programme, in two stages	4
1.3	Revision note	4
1.4	Contributions	5
1.5	What this note does not do	6
2	Why context-based full abstraction cannot help	6
2.1	An asymmetry between two translations	6
2.2	The door that stays open	6
2.3	Separation as the correctness frame	7
3	What unmodified rho already gives	7
3.1	The race equation	7
3.2	The residual is a which-path record	8
3.3	The normalisation fork	8
4	Two requirements, derived	9
4.1	Reconvergence must be counted over causal histories	9
4.2	The two syntactic requirements	9
5	The design	10
5.1	Resolution algebras	10
5.2	Two sorts of formula	10
5.3	Syntax	11
5.4	Per-match evaluation	11
5.5	The graded modality is a transfer operator	11
6	Formal semantics	12
6.1	Terms	12
6.2	The state space	12
6.3	Readings	13
7	Weight maps are a normal form	13

8	The join is the recombiner	14
8.1	Permutations of a join are the interfering pair	14
8.2	The minimal interferometer	15
9	A worked example: Shor’s algorithm	16
9.1	A qubit and a gate	16
9.2	Composition, and where the circuit’s wires went	17
9.3	The classical part is free	17
9.4	The term, in full	17
9.5	The instance, assembled	20
9.6	The hazard: unnormalised clauses buy post-selection	20
9.7	What the example shows, and what it does not	21
10	Dilation, heralds, and the price of erasure	21
10.1	The fork of §3.3 was not a trichotomy	21
10.2	The defect, dilated per site	22
10.3	The herald	23
10.4	Post-selection becomes a program	24
10.5	The price	24
10.6	Why not the linear fragment	25
11	PLN-valued where-clauses	26
11.1	Why this slot invites PLN	26
11.2	PLN truth values, briefly	26
11.3	The resolution algebra, honestly	27
11.4	Three regimes, one syntax	27
12	A second worked example: a mortal forager	28
12.1	The setting	28
12.2	What the graded clause replaces	29
12.3	The bootstrap trap	29
12.4	Two worlds, four foragers	29
12.5	Amortisation, measured	30
12.6	The personality parameter acquires a metabolic optimum	31
12.7	Up the fractal	31
13	Implementation	32
14	Open threads	32
15	Conclusion	34

1 Introduction

1.1 Where the choice is

A rho program does not determine its own execution. Write

$$x!(Q_1) \mid \text{for}(y \leftarrow x)P \mid x!(Q_2)$$

and the operational semantics permits two reductions. It does not say which one happens, and it does not say with what tendency. This is not an oversight; it is the defining feature of a

mobile process calculus, and it is what makes the calculus a specification of interaction rather than of a machine.

An implementation must close the gap. RSpace closes it by presenting the parallel fragment as a store from channels to polarity-homogeneous bags and letting the physical race between cores decide which produce is zipped with which consume. That is a resolution mechanism, and it is a choice — in the precise sense that the family of admissible pairings, indexed by contended channels, has a section only when something supplies one.

The observation this note develops is that the resolution mechanism is a *parameter*, and that other values of the parameter are available. In particular, quantum mechanics is a resolution mechanism: it supplies amplitudes for the branches of a race and the Born rule for turning them into outcomes. Instantiating rho’s parameter with that mechanism is a well-posed thing to do, and the question this note answers is what syntax a programmer needs in order to control it.

1.2 The programme, in two stages

The work proceeds in two stages, and it is worth separating them sharply because they have different success criteria.

Stage one exposes the additional structure. A quantum resolver distinguishes programs that classical observation identifies, because it permits interference. That is a failure of full abstraction — the denotation is finer than contextual equivalence — and the failure is the feature. The set of pairs that classical observation identifies and the quantum denotation separates is the specification document for stage two.

Stage two gives that structure syntactic expression, so that a programmer can write into the gap on purpose. Aaronson’s summary of quantum programming is apt here: most of the discipline consists of arranging for interference to do something useful. A syntax for arranging interference is therefore a syntax for exploring what quantum advantage is available, and having one is a precondition for a verdict rather than a consequence of one.

The weighted-GSLT programme proposed *weight maps* for stage two: a table, carried alongside the term, from spatial-behavioural formulae refining the left-hand side of a rewrite rule to weights. This note argues that the weight map had the right *language* and the wrong *location*, and that relocating it into the for-comprehension’s **where** clause — by the simple device of letting formulae take non-crisp truth values — both subsumes it and repairs three separate defects at once.

1.3 Revision note

This is the fourth version of the note. Sections 2–9 and §§11–12 are unchanged in substance. What is new is §10, and one correction it rests on.

The correction is to §3.3. That section offered three resolutions of the degenerate race and took the third, the unnormalised reading with the Born rule at observation. The third is not an alternative to the second: it is the second with the branch register traced out, and the trace is where the post-selection of §9.6 enters. Remark 10.1 states this. The presentation of §3.3 is left standing with a pointer, rather than rewritten, so that the reasoning which led to the defect remains legible.

What follows from it is that the defect wants a construction and not a side condition, and §10 supplies one. Definition 9.11 survives but is demoted: it no longer refuses programs, and Proposition 9.12 becomes a compiler optimisation rather than an admission test. Obligation 14.3 of the previous version, which asked for a static discipline enforcing isometry, is withdrawn and replaced.

1.4 Contributions

1. **A negative result about full abstraction** (§2). Enlarging the class of rho contexts cannot expose interference, because contexts can *create* apertures but cannot *read* them. The mismatch is in the observation type, not the context class, and the correct frame is a *separation* between resolvers rather than a full-abstraction theorem between terms. We give the replacement correctness condition: one-sided support adequacy, whose failure to be two-sided is the interference.
2. **A sharp stage-one verdict** (§3). Under the race equation with uniform weights, unmodified rho interferes exactly when the racing messages are structurally congruent, and then only constructively, because the surviving message is a which-path record. The phenomenon is a permanent; it is BosonSampling-shaped, hence plausibly hard and definitely not universal.
3. **The design** (§§5–6). Value the **where** clause in a commutative semiring and evaluate it per candidate match. Boolean values recover rholang 1.4 on the nose (Theorem 6.5).
4. **Weight maps are a normal form** (§7, Theorem 7.2). Every weight map is a graded clause of the shape $\bigoplus_i v_i \otimes [\chi_i]$, and the partition discipline exists solely to make that shape single-valued — a requirement a formula satisfies for free. Dynamic weight maps become formulae mentioning the current term, so the update function disappears too (Proposition 7.4).
5. **The join is the recombiner** (§8, Theorem 8.2). Which-path erasure — the half of the problem the weight map never addressed — requires no new term former. A join $\&$ over m patterns on a contended channel has $m!$ candidate matches whose outcomes coincide precisely when the continuation is symmetric in the bound names, and the graded clause supplies the $m!$ amplitudes. The clause is a scattering matrix and the symmetric outcome carries its permanent.
6. **Shor’s algorithm, and an overshoot** (§9). The fragment is universal for quantum circuits (Lemmas 9.2–9.4), the circuit’s DAG is the term’s causal order (Proposition 9.5), and the classical part costs nothing (Corollary 9.7). §9.4 gives the term in full rather than in lemmas, generated from the gate list the simulator runs. But with clauses unconstrained the fragment decides PP by post-selection (Proposition 9.10).
7. **Heralds, and the containment** (§10). The overshoot is closed by construction. Every receipt is dilated at the *site*, since Proposition 4.2 makes a global step the wrong unit, and the dilation’s ancilla is emitted as an ordinary message on a channel named by quoting the site. Proposition 10.10 shows the message may carry the defect bit and nothing else, on pain of destroying interference everywhere; Corollary 10.11 shows heralding is therefore inert exactly on the fragment Proposition 9.12 identifies, so the amendment can be uniform. Under it, post-selection is not a step (Proposition 10.13) but a retry loop, and the loop is charged: the free filter of §9.6 costs 2^n firings (Proposition 10.14, Conjecture 10.15).
8. **A negative result about linearity** (§10.6). Restricting to the fragment in which every bound name is used exactly once makes isometry structural, and is not the right move: the second erasure that Lemma 9.2 needs is bought by weakening, and a weakening-free step is block-diagonal in the input basis (Proposition 10.17), hence an isometry and not a gate. Theorem 8.2 is unaffected, since its continuations are already linear — the two erasures come apart, and only one of them is implicated.
9. **PLN-valued clauses** (§11). The same slot accepts Probabilistic Logic Network truth values, making inference and execution one loop. We give the algebra honestly, including

which of PLN’s operations supply values and which would need weakening to supply a semiring.

10. **A mortal forager** (§12). A second worked example, in the inference-driven regime. The complementary guard pair’s dependence on negation in the hypothesis language dissolves (Proposition 12.2); the power has a fixed point at zero, so a belief needs prior evidence or an exploration disjunct (Proposition 12.4); confidence is invisible without an outside option (Remark 12.3); and the personality parameter k acquires a metabolic optimum (Principle 12.6) — a return to PLN as well as from it.

1.5 What this note does not do

It does not prove a quantum advantage; it builds the instrument with which one could be sought. It does not prove Conjecture 10.15, whose soft point is named in Remark 10.16, and it does not develop the mode-encoded alternative of Remark 10.19. It does not treat replication or the full recursive fragment beyond noting where the budget must bite. And it does not attempt a categorical account of the construction, which we expect to be a Kleisli-style story over the semiring’s free-module monad and which we leave to a companion.

2 Why context-based full abstraction cannot help

2.1 An asymmetry between two translations

When one encodes the π -calculus into rho, a natural worry about full abstraction arises and a natural repair is available: rho contexts see strictly more than π contexts do, so demand preservation only against the images of π contexts. The repair is legitimate because the discrepancy it manages — rho’s ability to inspect the structure of a name — lies on an axis about which *both* calculi are silent in the same way, namely how a race is resolved. Restricting the context class is orthogonal to resolution, so it costs nothing on that axis.

The translation from rho into quantum mechanics is not like this. Quantum mechanics makes a definite commitment about resolution: amplitudes compose linearly and outcomes obey the Born rule. The discrepancy is therefore *on* the axis about which rho is silent. No restriction or enlargement of the class of rho contexts can address it, for the simple reason that a rho context is itself a rho term, whose own races are resolved by the same unspecified mechanism. Composing silence with silence yields silence.

Principle 2.1 (Contexts create apertures; they do not read them). A rho context can change which races exist — by adding contention, by consuming a contender, by installing a new receive. It cannot observe how any race is resolved. Consequently no property of the form “for all rho contexts $C[-], \dots$ ” can be sensitive to the resolution mechanism.

This is the precise reason a full-abstraction theorem is not the right correctness statement for the present setting, and the reason is stronger than the usual observation that interference makes the denotation too fine. It is not that we have too few contexts. It is that the discriminating power we want is not of the kind contexts have.

2.2 The door that stays open

The enterprise is not thereby vacuous, and it is worth being careful about why. A context cannot see a phase. But a context can be an *interferometer*: an arrangement in which a difference of phase becomes a difference in the *support* of the outcomes. This is how every physical quantum experiment reports its result — not by observing an amplitude but by a detector that never fires. §8 exhibits such a context in three lines of rholang.

What this does not do is rescue full abstraction, because the discrimination it achieves is between two *resolvers* at a fixed term, not between two terms at a fixed resolver.

2.3 Separation as the correctness frame

The right object of study is therefore the map

$$\text{Resolvers} \times \text{Terms} \longrightarrow \text{Distributions over normal forms,}$$

in which the classical RSpace semantics and the Born semantics are two points of the first coordinate. Quantum advantage is the assertion that the image of the quantum resolver is not contained in the image of the classical family — a separation, in the shape of a Bell argument, rather than a full-abstraction theorem.

What survives as a checkable correctness obligation is one-sided.

Definition 2.2 (Support adequacy and the interference deficit). Let $\text{Reach}(P) \subseteq \text{NF}$ be the set of normal forms operationally reachable from P , and let $\llbracket P \rrbracket$ be the graded denotation of §6. The denotation is *support-adequate* when

$$\text{supp}(\llbracket P \rrbracket) \subseteq \text{Reach}(P).$$

The *interference deficit* of P is $\mathcal{D}(P) := \text{Reach}(P) \setminus \text{supp}(\llbracket P \rrbracket)$.

Support adequacy says the interpretation invents nothing the operational semantics forbids. The converse inclusion is exactly what destructive interference violates: a normal form that some derivation reaches, whose amplitudes cancel. So the natural two-sided condition is not merely unproven, it is *false precisely where the interesting physics is*, and its failure is the observable. We will return to $\mathcal{D}$ as the quantity a programmer manipulates.

Remark 2.3 (Non-compositionality is not a defect). The store denotation of ρ is a monoid homomorphism for parallel composition, because it denotes the store. The sum-over-futures denotation cannot be, because composing P with Q creates races that neither had alone. This is not a weakness relative to quantum mechanics; it is the same phenomenon in the same place — interacting subsystems do not have product states. The cut is the interaction term, and entanglement across a cut is the semantic image of a race across a cut. It is also the semantic shadow of the observation that an aperture is a property of the cut and not of either agent flanking it.

3 What unmodified rho already gives

3.1 The race equation

Take as the constraint on an interpretation $\llbracket - \rrbracket$, at a two-way race between one receive and two sends,

$$\llbracket x!(Q_1) \mid \text{for}(y \leftarrow x)P \mid x!(Q_2) \rrbracket = w_1 \llbracket P\{\text{@}Q_1/y\} \mid x!(Q_2) \rrbracket + w_2 \llbracket x!(Q_1) \mid P\{\text{@}Q_2/y\} \rrbracket, \quad (1)$$

with $w_1 = w_2$ and the weights covering the total probability space; dually for two receives racing over one message. The constraint says nothing about a lone receive meeting a lone send. Alongside it we adopt the principle that the meaning of a term is the sum of all its possible futures.

Two things are worth noticing immediately about (1). It is *parameter-free*: the weights are forced by the arity of the race, so there is no table, no key language, and no partition. And it is *silent on the confluent fragment*, which localises whatever design freedom remains.

3.2 The residual is a which-path record

The left summand of (1) retains $x!(Q_2)$ and the right retains $x!(Q_1)$. That surviving message records which branch was taken. Two derivations whose residuals differ do not reconverge, and derivations that do not reconverge cannot interfere.

There is exactly one escape, and it is forced: when $Q_1 \equiv Q_2$ the record is vacuous, the two summands are structurally congruent, and the branches meet.

Proposition 3.1 (Stage-one verdict). *Under (1) with uniform real weights and no further apparatus, the only interference available to ρ is between derivations consuming structurally congruent messages, and it is always constructive.*

Proof sketch. Interference requires two derivations from a common source reaching a common target. At a single race the targets are $P\{@Q_1/y\} \mid x!(Q_2)$ and $x!(Q_1) \mid P\{@Q_2/y\}$, which are congruent only if $Q_1 \equiv Q_2$ (the substitution instances agreeing for all P forces the quoted payloads to agree). Iterating, any reconvergence decomposes into steps of this form together with permutations of independent redexes, which are identified by Definition 4.1 below. All weights being non-negative reals, summed amplitudes cannot cancel. $\square$

Remark 3.2 (The general race is a permanent). At a channel carrying a bag of produces and a bag of consumes, communication chooses an admissible pairing and zips, leaving surplus. Summing coherently over perfect matchings of an amplitude matrix is the computation of a permanent, which is to say that the coherent reading of the zip is BosonSampling-shaped: plausibly hard to sample, while the classical reading — pick a matching uniformly — is easy. The superlinear multiplicity that earlier work identified in the degenerate case is accordingly not a bookkeeping error but genuine indistinguishable-particle statistics.

The verdict is therefore genuinely good news for the design, because it makes the brief precise: what ρ lacks is not interference *per se* but *phase* and *control over recombination*. The analogy with linear optics is close enough to be useful. Indistinguishable bosons through a passive network give hardness without universality; adding phase control and adaptivity is what carries one to KLM-style universality. The gap to be closed may be small.

3.3 The normalisation fork

Equation (1) is not consistent as stated, and the inconsistency shows up at its own simplest instance. With $Q_1 \equiv Q_2$ the right-hand side is $(w_1 + w_2)\llbracket T \rrbracket$ for a single T . If the w_i are amplitudes with $|w_1|^2 + |w_2|^2 = 1$, this has norm $\sqrt{2}$; for an m -fold degenerate race, norm $\sqrt{m}$ and weight m .

Three resolutions are available.

1. *Convex.* Let $\llbracket - \rrbracket$ take values in formal convex combinations, $w_i = 1/m$. Consistent, and exactly the classical scheduler with a fair coin. No interference.
2. *Branch register.* Adjoin the derivation index so the summands stay distinguishable. Consistent and interference-capable, at the cost of carrying the index — this is the route taken by the ZX compilation work.
3. *Unnormalised.* Let $\llbracket - \rrbracket$ take values in the free $\mathbb{V}$ -module on normal forms and apply the Born rule at observation. Consistent, with m -fold degeneracy carrying weight m^2 .

Version 4 note. The trichotomy is not one; see Remark 10.1. The third option is the second with the register traced out, and §10 replaces the choice below with a construction. The paragraph is left as first written because the defect it introduces is instructive.

We take the third, on the ground that linear-in- m and quadratic-in- m are the values of two different observables rather than two normalisations of one: the first is what one gets by looking at which derivation fired, the second by declining to look. Since the whole point of the exercise is to decline to look, the quadratic answer is the correct one here, and it is exactly the bosonic enhancement of Remark 3.2.

4 Two requirements, derived

Interference needs two things and nothing else: derivations that *reconverge*, and a nonzero *relative phase* between them. Every syntactic feature proposed below is answerable to one of these.

4.1 Reconvergence must be counted over causal histories

A caution first, because it disciplines the path sum. Every concurrency diamond is a reconvergence: two independent redexes fired in either order reach the same term. Rho programs are saturated with these. If the sum over futures ranges over *interleavings*, then m independent redexes contribute a factor of $m!$ arising from nothing but the number of linearisations of a partial order.

Definition 4.1 (Independence and causal histories). Two redex firings at a term are *independent* when their footprints — the data and continuations they consume — are disjoint. Let $\sim$ be the least congruence on derivations identifying $u; v$ with $v; u$ for independent u, v . A *causal history* is a $\sim$ -class of derivations, and the path sum of §6.3 ranges over causal histories.

Proposition 4.2 (No spurious combinatorial enhancement). *Summing over causal histories rather than interleavings removes the $m!$ factor, and does not remove any reconvergence between causally inequivalent derivations.*

The apparatus for this choice already exists in the history-monad account of rho: free histories form the universal cover of the reduction graph and the erasure lattice is indexed by subgroups of its fundamental group. Filling the concurrency squares is one such erasure, and interference lives on the loops that survive it.

4.2 The two syntactic requirements

- (R1) **Phase conditional on data.** The amplitude attaching to a branch must be allowed to depend on which datum that branch consumed, and on the structure of the term around it.
- (R2) **Which-path erasure.** The programmer must be able to arrange that two branches produce the *same* residual, so that their amplitudes are added rather than kept apart.

Remark 4.3 (Auditing weight maps against the requirements). A weight map answers (R1) well: its keys are spatial-behavioural formulae refining the left-hand side of a rule, which is precisely “a phase conditional on the structure of what was matched.” It says nothing whatever about (R2); in the ZX treatment, recombination arrives as a choice of measurement basis at the end, which is an observer’s decision made outside the program. And it is ergonomically misplaced: a table carried in the state is a Hamiltonian specification, whereas arranging interference is done with term formers, at the site where the branching happens.

Algebra	$\oplus, \otimes$	Resolver	Regime
$\mathbb{B} = (\{0, 1\}, \vee, \wedge)$	$\vee, \wedge$	demonic / RSpace race	rholang 1.4 exactly
$\mathbb{R}_{\geq 0} = (\mathbb{R}_{\geq 0}, +, \times)$	$+, \times$	Gillespie / SSA	stochastic
$\mathbb{C} = (\mathbb{C}, +, \times)$	$+, \times$	Born rule	interference
Viterbi $([0, 1], \max, \times)$	$\max, \times$	greedy	most-likely path
Tropical $(\mathbb{R} \cup \{\infty\}, \min, +)$	$\min, +$	least cost	cost-optimal path
pow-PLN $([0, 1], +, \times)$	$+, \times$	inference-driven	§11

Table 1: Instances. One syntax; the resolver is the choice of $\mathbb{V}$.

5 The design

The design is one change, stated in a sentence: *the **where** clause of the for-comprehension is valued in a commutative semiring and is evaluated once per candidate match.*

5.1 Resolution algebras

Definition 5.1 (Resolution algebra). A *resolution algebra* is a commutative semiring $\mathbb{V} = (V, \oplus, \otimes, 0, 1)$: $\oplus$ associative and commutative with unit 0, $\otimes$ associative and commutative with unit 1, $\otimes$ distributing over $\oplus$, and 0 annihilating.

The two operations are forced by what the connectives must mean.

- *Conjunction* is $\otimes$. Two conditions holding together compose multiplicatively; this is also what makes the joined receive $p_1 \leftarrow c_1$ & $p_2 \leftarrow c_2$ combine its clauses correctly.
- *Disjunction* is $\oplus$. Two ways for a match to be enabled add.

Principle 5.2 (Interference is a logical phenomenon). Since $\llbracket \varphi \vee \psi \rrbracket = \llbracket \varphi \rrbracket \oplus \llbracket \psi \rrbracket$, in a resolution algebra with additive inverses a disjunction may evaluate to 0 though neither disjunct does. Destructive interference is a *false disjunction with true disjuncts*. This is the whole of the additional structure, stated in the logic rather than imposed on it from outside.

Negation is the casualty and it is worth saying so plainly. On $[0, 1]$ the map $v \mapsto 1 - v$ serves; on $\mathbb{C}$ there is no canonical complement and De Morgan fails against $(+, \times)$. The design therefore keeps two sorts, with the crisp sort embedded in the graded one.

5.2 Two sorts of formula

Let the crisp sort be the existing condition language of rholang 1.4: ground formulae supplied by a language definition, the spatial connectives generated from the term formers, the context-labelled modality $\langle K \rangle$, and full Boolean structure.

$$\beta ::= \text{true} \mid \text{false} \mid g \mid \neg\beta \mid \beta \wedge \beta \mid \beta \vee \beta \mid \langle K \rangle \beta \mid \text{out}(n)\beta \mid \text{in}(n)\beta \mid \beta \mid \beta \mid @\beta$$

and let the graded sort be

$$\psi ::= [\beta] \mid v \mid \psi \otimes \psi \mid \psi \oplus \psi \mid \langle K \rangle_{\mathbb{V}} \psi \quad (v \in V).$$

Here $[\beta]$ is the indicator embedding, sending a satisfied β to 1 and an unsatisfied one to 0, and v ranges over scalar constants of the algebra. The graded sort has no negation and no fixed-point operator; the crisp sort keeps both.

Remark 5.3 (One slot, not two). It is tempting to give the for-comprehension two clauses, a Boolean gate and a graded weight. It is better to give it one graded clause and recover the gate as its *support*: a communication is enabled exactly when its clause evaluates to something other than 0. This keeps the language unforked — existing Boolean where-clauses are the $\mathbb{B}$ -valued fragment under $\mathbb{B} \hookrightarrow \mathbb{V}$ — and it makes support adequacy (Definition 2.2) hold by construction rather than by proof, since the classical interpreter sees nothing of the value except whether it vanishes.

5.3 Syntax

The for-comprehension of rholang 1.4 already carries a **where** clause, whose condition may mention the variables bound by the preceding receipts. The change is only to the sort of that condition.

```
for( p11 <- c11 & ... & pm1 <- cm1 ;
    ... ;
    p1n <- c1n & ... & pmn <- cmn
    where psi ) P
```

with **psi** a graded formula over the algebra $\mathbb{V}$ declared for the shard or the channel's language definition. Within **psi**, the bound names p_{ij} are in scope, so the value may depend on what was matched — which is requirement (R1).

5.4 Per-match evaluation

The decisive point, and the one from which everything in §8 follows, concerns *when* the clause is evaluated.

Definition 5.4 (Candidates). Let r be a receipt site in P — a for-comprehension together with the channels its receipts name. A *candidate* at r is an assignment σ of a distinct datum occurrence to each pattern of each group, such that every pattern matches its assigned datum. Write $\text{Cand}(P, r)$ for the set of candidates and $\text{out}(r, \sigma) \in \mathcal{P}/\equiv$ for the residual term the firing of r under σ produces.

Candidates are indexed by *occurrences*, not by terms. This matters: two structurally congruent messages on a channel are two candidates, not one, because parallel composition is a multiset and structural congruence does not delete duplicates. An implementation that keys data by content alone, or that allocates identifiers during firing, cannot represent Cand faithfully.

Definition 5.5 (Graded clause valuation). A graded clause ψ at a receipt site r denotes a function

$$\llbracket \psi \rrbracket_r : \text{Cand}(P, r) \longrightarrow V,$$

defined by structural recursion, with $\llbracket \beta \rrbracket$ evaluated against the term P under the substitution σ carries, scalars constant, $\otimes$ and $\oplus$ pointwise, and $\langle K \rangle_{\mathbb{V}}$ as in §5.5.

So a clause is a *vector* indexed by candidates, and — once we compose it with out — a *matrix* over candidates and outcomes. That is the whole mechanism.

5.5 The graded modality is a transfer operator

Let $\mathbb{V}\langle \mathcal{P}/\equiv \rangle$ be the free $\mathbb{V}$ -module on terms modulo structural congruence. Define the one-step operator of a term by summing its candidates:

$$T(P) = \sum_r \sum_{\sigma \in \text{Cand}(P, r)} \llbracket \psi_r \rrbracket_r(\sigma) \cdot \text{out}(r, \sigma), \quad (2)$$

the outer sum over receipt sites, taken over causal histories in the sense of Definition 4.1, and the coefficient arithmetic in $\mathbb{V}$. Then the graded modality is

$$\llbracket \langle K \rangle_{\mathbb{V}} \psi \rrbracket(P) = \sum_{r, \sigma} \llbracket \psi_r \rrbracket_r(\sigma) \otimes \llbracket \psi \rrbracket(\text{out}(r, \sigma)),$$

which is T acting on the vector of values of ψ . In other words the graded modality is the transfer matrix of the reduction relation, and the principle that the meaning of a term is the sum of all its possible futures is exactly the statement that $\llbracket - \rrbracket$ is a fixed point of $\langle K \rangle_{\mathbb{V}}$. The denotational principle and the graded logic are one object seen twice.

6 Formal semantics

6.1 Terms

We work with core rho, taken with a built-in name restriction whose desugaring is not our concern here.

$$P ::= 0 \mid x!(P) \mid \text{for}(\vec{y} \leftarrow \vec{x} \text{ where } \psi)P \mid P \mid P \mid *x \mid \text{new } x \text{ in } P, \quad x ::= @P.$$

Quoting is $@P$ and dropping is $*x$. Structural congruence $\equiv$ is the least congruence making $(\mathcal{P}, |, 0)$ a commutative monoid, together with the usual laws for **new** and $*@P \equiv P$.

Remark 6.1 (Receipts are linear). Throughout, $\leftarrow$ is the *linear* receipt: firing consumes the continuation as well as the datum. Rholang also offers a persistent receipt, written $\leftarrow=$, which is not consumed. Linear is the default here, and in one place the choice is load-bearing rather than stylistic.

Under a persistent receipt the term of §3 is not a race at all. Both messages are eventually consumed, the receipt survives in either branch, and the two derivations reconverge on $P\{\@Q_1/y\} \mid P\{\@Q_2/y\} \mid \text{for}(y \leftarrow x)P$. The two firings consume disjoint data and no shared continuation, so they are independent in the sense of Definition 4.1: the square is filled and there is nothing left to weight. Equation (1)’s w_1, w_2 would then be describing an *ordering* rather than a choice. It is linearity that makes the surviving message a which-path record, and so linearity that Proposition 3.1 rests on.

Elsewhere persistence is harmless but idle. A persistent join still consumes all the data at its channel, and the surviving receipt is identical across candidates, so it distinguishes none of them; Theorem 8.2 goes through unchanged and the site is merely left inert with nothing further to receive.

6.2 The state space

Definition 6.2 (Graded denotation). Fix a resolution algebra $\mathbb{V}$ and a budget $\beta \in \mathbb{N}$. Let $M := \mathbb{V}\langle \mathcal{P} / \equiv \rangle$ be the free $\mathbb{V}$ -module on terms modulo structural congruence, with basis vectors written $|P\rangle$. The β -budgeted denotation is

$$\llbracket P \rrbracket_{\beta} := T_{*}^{\beta} |P\rangle = \sum_{h: P \Rightarrow^{\leq \beta} Q} w(h) |Q\rangle,$$

where T_{*} extends T of (2) linearly, h ranges over causal histories of length at most β ending at a normal form or exhausting the budget, and $w(h)$ is the $\otimes$ -product of the clause values along h .

Remark 6.3 (Why a budget, and only one). The definition is corecursive and does not bottom out on non-terminating terms. For terminating P the recursion is well-founded on the reduction

tree and β may be taken to be its depth. In general one needs either a convergence hypothesis — a spectral radius condition on T , since $\mathbb{C}$ has no order and Knaster–Tarski is unavailable — or a budget. We take the budget, and note it is the same budget already charged elsewhere: the risk ledger’s single ω for reflection, the address-space bound of the compilation route, and the inference-token budget that keeps generated model-checking terminating. These should be one budget doing three jobs rather than three budgets.

To make normal forms absorbing and keep histories comparable in length we adopt the usual convention.

Definition 6.4 (Fictitious jump). At a normal form Q the operator acts as $T|Q\rangle = |Q\rangle$.

6.3 Readings

One denotation admits three readings, differing only in what is done at the end.

1. *Support*. $\text{supp}(\llbracket P \rrbracket_\beta) \subseteq \text{NF}$. This is what the classical interpreter sees, and by Remark 5.3 it is all it sees.
2. *Born*. For $\mathbb{V} = \mathbb{C}$, the distribution $Q \mapsto |\langle Q | \llbracket P \rrbracket_\beta \rangle|^2$, normalised.
3. *Stochastic*. For $\mathbb{V} = \mathbb{R}_{\geq 0}$, the same expression without the square, which is the jump-chain marginal of the Gillespie reading.

Theorem 6.5 (Conservativity). *Let $\mathbb{V} = \mathbb{B}$ and let every clause be the embedding $[\beta]$ of a crisp condition. Then $\text{supp}(\llbracket P \rrbracket_\beta)$ is exactly the set of normal forms reachable from P in at most β steps under rholang 1.4’s operational semantics with its Boolean *where* clauses.*

Proof sketch. In $\mathbb{B}$, $\otimes = \wedge$ and $\oplus = \vee$, so $w(h) = 1$ iff every step of h has a satisfied clause, and the coefficient of $|Q\rangle$ is the disjunction over histories reaching Q . Candidates coincide with the admissible zips of the store presentation, and the fictitious jump does not add normal forms. Hence the coefficient is 1 iff some admissible derivation reaches Q . $\square$

Corollary 6.6 (Support adequacy). *For any $\mathbb{V}$, $\text{supp}(\llbracket P \rrbracket_\beta) \subseteq \text{Reach}_\beta(P)$, since a nonzero coefficient requires at least one history with all clause values nonzero, and such a history is admissible under the Boolean image of the clauses. Equality fails exactly when some Reach-witnessed coefficient is a sum of nonzero terms summing to 0 — that is, exactly on the interference deficit $\mathcal{D}(P)$.*

This is the payoff of Remark 5.3. Adding grading cannot make a classical program do something it could not do; it can only make it fail to do something it could.

7 Weight maps are a normal form

Definition 7.1 (Weight map, recalled). A weight map for a rewrite rule R is a finite family of spatial-behavioural formulae $\chi_1, \dots, \chi_k$ refining the left-hand side of R , required to *partition* that refinement, together with values $v_1, \dots, v_k \in V$; the weight of a redex is v_i for the unique i with χ_i satisfied.

Theorem 7.2 (Weight maps are the linear graded clauses). *Every weight map $(\chi_i, v_i)_{i \leq k}$ is denoted by the graded clause*

$$\psi = \bigoplus_{i=1}^k v_i \otimes [\chi_i],$$

and conversely a graded clause of this shape denotes a weight map if and only if the χ_i partition the refinement. Graded clauses not of this shape — those with nested $\otimes$ over overlapping conditions, or with a graded modality — are strictly more expressive.

Proof sketch. Forwards: on a candidate satisfying χ_j and no other, all but the j th summand vanish and the value is v_j , and totality of the partition ensures some summand survives. Backwards: if two χ 's overlap on a candidate, the clause returns $v_i \oplus v_j$, which is a well-defined value but not a table lookup; the partition condition is exactly the condition making the sum a selection. Strictness: with overlapping χ_i the clause is a genuine mixture no weight map assigns, and $\langle K \rangle_{\forall}$ inspects successors, which no key can. $\square$

Corollary 7.3 (The partition discipline is unnecessary). *Single-valuedness of a graded clause is automatic, a formula having one value. The requirements that key sets partition the left-hand-side refinement, that partitions be constructed through a single checked funnel, and that a default class complete them, all exist to secure single-valuedness for a lookup and have no analogue here.*

This also dissolves a specific obstruction. Earlier work observed a tension between the *Boolean demand* — a weight map's partition needs complements, so the target logic specification must supply negation — and the fact that the transport of a generated logic along a bisimulation-preserving translation preserves only the geometric fragment, which has no complements. With no partition to define, the Boolean demand does not arise. Negation is still absent from the graded sort, but it is absent as a design choice about the value algebra rather than as an unmet requirement.

Proposition 7.4 (Plasticity is free). *A dynamic weight map, whose table is revised after each rewrite by an update function $u : W \times \text{Match} \times \text{Trace} \rightarrow W$, is denoted by a static graded clause whenever u 's dependence on the trace is expressible as a crisp formula over the current term. No update function is needed, because a clause is re-evaluated against the current term at every candidate.*

Proof sketch. The clause ψ mentions the term through its crisp subformulae; its value at a candidate in the term P' reached after some history is by definition computed against P' . A table revised to depend on features of P' therefore agrees with the clause $\bigoplus_i v_i \otimes [\chi_i]$ in which the χ_i mention those features directly. Dependence on trace history beyond what is visible in P' is not expressible, which is the hypothesis. $\square$

Remark 7.5. Proposition 7.4 retires a specific implementation obligation. The plan for compiling the weighted simulator identified the update function — an arbitrary closure over time and a trace summary — as the one component with no compilable form, and proposed a declarative sub-language for it. The graded clause *is* that sub-language, and it was already needed for other reasons. The restriction to features visible in the current term has precedent: the exhaustive-graph construction already refuses time-dependent updates by fixing the clock at zero.

8 The join is the recombiner

Requirement (R2) asked for which-path erasure, and the audit of Remark 4.3 found nothing in the weight map that supplies it. It turns out no new term former is needed: the erasure is already in the language, in the join.

8.1 Permutations of a join are the interfering pair

Consider a receipt whose group contains m patterns on a single contended channel, meeting m data occurrences:

$$x!(Q_1) \mid \cdots \mid x!(Q_m) \mid \text{for}(y_1 \leftarrow x \ \& \ \cdots \ \& \ y_m \leftarrow x \ \text{where } \psi)P.$$

The candidates are the $m!$ bijections σ from patterns to data. Every candidate consumes *all* the data — there is no surplus, hence no residual message and no which-path record — and produces $\text{out}(\sigma) = P\{\text{@}Q_{\sigma(1)}/y_1, \dots, \text{@}Q_{\sigma(m)}/y_m\}$.

Definition 8.1 (Symmetric continuation). P is *symmetric* in $y_1, \dots, y_m$ when

$$P\{\text{@}Q_{\sigma(1)}/y_1, \dots\} \equiv P\{\text{@}Q_{\tau(1)}/y_1, \dots\}$$

for all bijections σ, τ and all payloads.

Symmetry is cheap to arrange, because parallel composition is commutative: the continuation $z!(\ast y_1) \mid z!(\ast y_2)$ is symmetric, while $z_1!(\ast y_1) \mid z_2!(\ast y_2)$ is not.

Theorem 8.2 (The clause is a scattering matrix). *Let $A_{ij} := \llbracket \psi \rrbracket(y_i \mapsto \text{@}Q_j)$ be the value of the clause on the candidate assigning datum j to pattern i , and suppose ψ factors as $\bigotimes_i \psi_i(y_i)$ over patterns. If P is symmetric in the y_i , all $m!$ candidates share a single outcome $\bar{P}$, and*

$$\langle \bar{P} \mid T \mid P\text{-site} \rangle = \sum_{\sigma \in S_m} \prod_{i=1}^m A_{i\sigma(i)} = \text{Per}(A).$$

If P is not symmetric, the candidates have pairwise distinct outcomes and each carries its own product with no summation.

Proof sketch. Every candidate is a bijection and the clause factors, so the weight of the candidate σ is $\prod_i A_{i\sigma(i)}$. Under symmetry all outcomes are one basis vector and the coefficients add, which is the permanent. Without symmetry the outcomes are distinct basis vectors and no addition occurs. $\square$

Corollary 8.3 (Which-path, syntactically). *No choice of clause produces interference at a receipt whose continuation distinguishes its bound names. Whether a race interferes is decided by the continuation, not by the weights.*

This is the design’s central ergonomic fact. The programmer has two dials at exactly the site where the branching happens: the clause sets the amplitudes, and the continuation decides whether the branches are told apart. Arranging interference is arranging for the continuation to forget.

Remark 8.4 (Statistics). $\text{Per}(A)$ is the bosonic amplitude. A clause assigning the sign of σ — expressible when the algebra has additive inverses and the crisp sort can detect the relevant order — gives $\text{Det}(A)$ and hence fermionic statistics, in which coincident data annihilate. That the same syntactic slot produces both is a point in favour of the slot.

8.2 The minimal interferometer

Take $\mathbb{V} = \mathbb{C}$, $m = 2$, and a clause that returns a on the identity assignment and b on the transposition. With a symmetric continuation:

```
new x, z in {
  x!(Q1) | x!(Q2) |
  for( y1 <- x & y2 <- x where psi ) { z!(\ast y1) | z!(\ast y2) }
}
```

The single outcome $z!(\ast Q_1) \mid z!(\ast Q_2)$ carries amplitude $a + b$. Choosing $b = -a$ makes it zero. That normal form is operationally reachable — indeed it is the *only* reachable normal form — so $\mathcal{D}(P) \neq \emptyset$ and Corollary 6.6’s inclusion is strict. Replacing the continuation by

$z_1!(*y_1) \mid z_2!(*y_2)$ restores two distinct outcomes with amplitudes a and b , probabilities $|a|^2$ and $|b|^2$, and no cancellation whatever the phases.

Three lines of rholang thus exhibit the entire phenomenon: a detector that never fires, and a which-path measurement that destroys the effect. It is the Hong–Ou–Mandel arrangement, with the channel z playing the role of a single detector and the pair z_1, z_2 of two.

Remark 8.5 (What a rho context can and cannot do). Note the consistency with §2. The context $\text{for}(y_1 \leftarrow x \ \& \ y_2 \leftarrow x \dots) \{z!(*y_1) \mid z!(*y_2)\}$ does not observe the resolution of the race; it *arranges* the race so that resolution leaves a trace in the support. This is precisely what was meant by a context acting as an interferometer, and it is the reason the enterprise has observable consequences without any context ever reading an amplitude.

9 A worked example: Shor’s algorithm

The design of §5 was derived from what interference requires, not from the circuit model, so it is a fair question whether it reaches the standard algorithms. It does. The instructive part is not that the fragment turns out expressive enough but that it turns out *too* expressive, which §9.6 addresses and which changes the status of one of this note’s open threads.

Every number reported in this section is computed by a simulator implementing Definition 6.2 literally: it enumerates the candidates at each site and adds coefficients on coinciding outcomes. Nothing in it multiplies gate matrices. Matrices appear only as the values a clause returns on a candidate.

9.1 A qubit and a gate

Definition 9.1 (Wire encoding). A *qubit* is a channel carrying exactly one of two designated data. Writing a_0, a_1 for the two, the basis states are $q!(a_0)$ and $q!(a_1)$.

```
new q, w, q' in {
  q!(*in) | w!(a0) | w!(a1) |
  for( v <- q & u1 <- w & u2 <- w where psi ) { q'!(u1) }
}
```

Lemma 9.2 (Arbitrary one-qubit gate). *The site above has exactly two candidates, the bijections $\{u_1, u_2\} \rightarrow \{a_0, a_1\}$. Each consumes every datum on q and on w , so no residual records which fired. The outcomes are $q'!(a_0)$ and $q'!(a_1)$; they are distinct, and neither mentions v , hence they are the same two terms for either input encoding. Therefore the one-step operator restricted to $\text{span}\{q!(a_0), q!(a_1)\}$ is the matrix $A_{ij} = \llbracket \psi \rrbracket(v \mapsto a_i, u_1 \mapsto a_j)$, and since ψ may mention both v and u_1 , every 2×2 matrix over $\mathbb{V}$ is realised.*

Remark 9.3 (Two erasures, not one). Theorem 8.2 identified outcomes of *different candidates at one site*, which is recombination within a superposition. A gate needs the other identification: outcomes at *different input basis states* must coincide, which is what makes the site a linear map into a fixed codomain rather than a family of unrelated branchings. Both are bought by the same device — a join consumes everything, and the continuation re-emits without mentioning what was consumed. That the join’s totality purchases both erasures is the reason no further term former is needed here either.

Lemma 9.4 (Arbitrary two-qubit gate). *Joining two input channels with two ancilla channels gives $2 \times 2 = 4$ candidates and four distinct outcomes indexed by the pair of u_1 -assignments. The clause may mention both input data and both assignments, so every 4×4 matrix is realised, including entangling ones. Diagonal gates are the case where the clause vanishes off the diagonal, and by Remark 5.3 those candidates simply do not fire.*

9.2 Composition, and where the circuit's wires went

Proposition 9.5 (The circuit's DAG is the term's causal order). *A gadget is enabled exactly when its input channels carry data. Composing gadgets in series on fresh channels therefore makes the data dependency of the term equal to the wire structure of the circuit, and every reduction order is a linear extension of the circuit's dependency DAG. No scheduling discipline is required and none is expressible: the term does not encode the circuit's order, it is that order.*

Proposition 4.2 is load-bearing here rather than hygienic. Gates in one circuit layer are independent redexes, so without the causal quotient a layer of n gates would contribute a factor $n!$ from counting linearisations. Measured, for n independent one-qubit gadgets: the causal sum gives squared norm 1.000000 for $n = 2, 3, 4$, while summing over interleavings inflates it by exactly 2, 6 and 24.

9.3 The classical part is free

Proposition 9.6 (Deterministic communication is amplitude-transparent). *A communication with a single candidate contributes the clause's value on that candidate, which for a crisp clause is 1. Hence amplitudes are unchanged by deterministic reduction, and the length of a derivation does not affect its weight.*

Corollary 9.7. *Any ordinary rho program runs inside every branch of a superposition at no cost in the semantics. In particular modular exponentiation — the expensive part of Shor's algorithm, and the part usually requiring laborious reversible compilation — is written as ordinary rholang.*

Measured: the block computing $x \mapsto 7^x \bmod 15$ carries a 16-term superposition to a 16-term superposition with norm 1.000000000000, exactly preserved.

9.4 The term, in full

The gadgets above are stated as lemmas, and lemmas are easy to believe and hard to run. This subsection gives the whole program. It is not transcribed: the listings below are emitted by `shorterm.py` from the same gate list that `gradedsim.py` executes to produce Table 2, so the code and the numbers cannot drift apart. Running the emitted list reproduces the distribution of Table 2 with a worst deviation of 0.

Three conventions. A **contract** is a persistent receipt on an installer channel; the receipt it installs on a wire is linear, so by Remark 6.1 the persistence never touches a race. Crisp guards are written in brackets and multiply graded scalars, which is the normal form of Theorem 7.2; **cis** denotes $\theta \mapsto e^{i\theta}$, a scalar constant of $\mathbb{V} = \mathbb{C}$. Channels are passed as data in the usual way, so an argument is dereferenced where it is used as a channel.

```
contract gate1( qIn, qOut, m00, m01, m10, m11 ) = {
  new w in {
    w!(0) | w!(1)
    | for( v <- *qIn & u1 <- w & u2 <- w
      where [ *v == 0 and *u1 == 0 ] * *m00
          + [ *v == 0 and *u1 == 1 ] * *m01
          + [ *v == 1 and *u1 == 0 ] * *m10
          + [ *v == 1 and *u1 == 1 ] * *m11 )
      { *qOut!(*u1) }
  }
}

contract hadamard( qIn, qOut ) = {
```

```

    gate1!( *qIn, *qOut,  0.7071067811865476,  0.7071067811865476,
              0.7071067811865476, -0.7071067811865476 )
}

contract cphase( qaIn, qbIn, qaOut, qbOut, theta ) = {
  new wa, wb in {
    wa!(0) | wa!(1) | wb!(0) | wb!(1)
    | for( va <- *qaIn & vb <- *qbIn
      & ua1 <- wa & ua2 <- wa & ub1 <- wb & ub2 <- wb
      where [ *ua1 == *va and *ub1 == *vb ]
        * ( [ *va == 1 and *vb == 1 ] * cis(*theta)
          + [ not (*va == 1 and *vb == 1) ] * 1 ) )
      { *qaOut!(*ua1) | *qbOut!(*ub1) }
    }
  }
}

contract swap( qaIn, qbIn, qaOut, qbOut ) = {
  new wa, wb in {
    wa!(0) | wa!(1) | wb!(0) | wb!(1)
    | for( va <- *qaIn & vb <- *qbIn
      & ua1 <- wa & ua2 <- wa & ub1 <- wb & ub2 <- wb
      where [ *ua1 == *vb and *ub1 == *va ] * 1 )
      { *qaOut!(*ua1) | *qbOut!(*ub1) }
    }
  }
}

// Modular exponentiation: one candidate at every step, clause value 1.
contract modexp( ret, base, expn, modulus ) = {
  new loop in {
    contract loop( acc, b, e ) = {
      if ( *e == 0 ) { *ret!(*acc) }
      else {
        if ( *e % 2 == 1 ) {
          loop!( (*acc * *b) % *modulus, (*b * *b) % *modulus, *e / 2 )
        } else {
          loop!( *acc, (*b * *b) % *modulus, *e / 2 )
        }
      }
    }
  }
  | loop!( 1, *base % *modulus, *expn )
}
}

```

The instance follows. Every wire carries an explicit stage index, because gadgets compose in series on *fresh* channels (Proposition 9.5): $q3_5$ is the fourth wire after five gates. The indices are the circuit's wires, drawn out.

```

new q0_0, q0_1, q0_2, q0_3, q0_4, q0_5, q0_6, q0_7, q1_0, q1_1, q1_2,
  q1_3, q1_4, q1_5, q1_6, q1_7, q2_0, q2_1, q2_2, q2_3, q2_4, q2_5,
  q2_6, q2_7, q3_0, q3_1, q3_2, q3_3, q3_4, q3_5, q3_6, q3_7, q4_0,
  q4_1, q5_0, q5_1, q6_0, q6_1, q7_0, q7_1, done in {

  // 1. the input register
  q0_0!(0) |
  q1_0!(0) |
  q2_0!(0) |
  q3_0!(0) |

```

```

q4_0!(0) |
q5_0!(0) |
q6_0!(0) |
q7_0!(1) |

hadamard!( *q0_0, *q0_1 ) |
hadamard!( *q1_0, *q1_1 ) |
hadamard!( *q2_0, *q2_1 ) |
hadamard!( *q3_0, *q3_1 ) |

// 2. modular exponentiation, entangling work with counting
//   y0..y3 are discarded: the work register is overwritten
for( x0 <- *q0_1 & x1 <- *q1_1 & x2 <- *q2_1 & x3 <- *q3_1
    & y0 <- *q4_0 & y1 <- *q5_0 & y2 <- *q6_0 & y3 <- *q7_0 ) {
  new r in {
    modexp!( *r, 7, 8 * *x0 + 4 * *x1 + 2 * *x2 + *x3, 15 ) |
    for( m <- r ) {
      q0_2!(*x0) | q1_2!(*x1) | q2_2!(*x2) | q3_2!(*x3) |
      q4_1!( (*m / 8) % 2 ) | q5_1!( (*m / 4) % 2 ) |
      q6_1!( (*m / 2) % 2 ) | q7_1!( *m % 2 )
    }
  }
} |

// 3. the transform on the counting register
hadamard!( *q0_2, *q0_3 ) |
cphase!( *q1_2, *q0_3, *q1_3, *q0_4, 2*pi/4 ) |
cphase!( *q2_2, *q0_4, *q2_3, *q0_5, 2*pi/8 ) |
cphase!( *q3_2, *q0_5, *q3_3, *q0_6, 2*pi/16 ) |
hadamard!( *q1_3, *q1_4 ) |
cphase!( *q2_3, *q1_4, *q2_4, *q1_5, 2*pi/4 ) |
cphase!( *q3_3, *q1_5, *q3_4, *q1_6, 2*pi/8 ) |
hadamard!( *q2_4, *q2_5 ) |
cphase!( *q3_4, *q2_5, *q3_5, *q2_6, 2*pi/4 ) |
hadamard!( *q3_5, *q3_6 ) |
swap!( *q0_6, *q3_6, *q0_7, *q3_7 ) |
swap!( *q1_6, *q2_6, *q1_7, *q2_7 ) |

// 4. read the counting register
for( c0 <- *q0_7 & c1 <- *q1_7 & c2 <- *q2_7 & c3 <- *q3_7 ) {
  done!( 8 * *c0 + 4 * *c1 + 2 * *c2 + *c3 )
}
}

```

Remark 9.8 (What the listing exposes). Three things are visible here that the lemmas state but do not show. The first is that the whole quantum content of the program sits in the **where** clauses of three contracts, and everything else is rholang. The second is that the ancilla patterns `u2`, `ua2`, `ub2` are bound and never used, and neither are `y0–y3` in the modular exponentiation step; those four discards are the weakening that §10.6 shows the construction cannot do without. The third is that the term contains no scheduling of any kind. The order of the transform is carried entirely by the stage indices — each gadget waits for its input channel — so the twelve lines of §§2–3 may be written in any order at all without changing the program.

9.5 The instance, assembled

Theorem 9.9 (Expressibility). *For N with $n = \lceil \log_2 N \rceil$, Shor’s algorithm is a graded rho term using $O(n^2)$ gadgets of arity at most two for the quantum Fourier transform, $O(n^3)$ deterministic communications for modular exponentiation, and a term size and reduction depth polynomial in n .*

Two verifications.

The transform. The QFT assembled from the gadgets of Lemmas 9.2–9.4 — Hadamards, controlled phases and swaps, in the standard decomposition — was compared entry by entry against the discrete Fourier matrix $\omega^{jk}/\sqrt{2^n}$. Worst entry error: 2.7×10^{-16} at $n = 2$, 1.4×10^{-15} at $n = 3$, 3.8×10^{-15} at $n = 4$.

The instance. $N = 15$, $a = 7$ (so $r = 4$), with four counting and four work qubits. The run fires 8 one-qubit and 8 two-qubit gadgets, evaluating 390 candidates in all, and ends with 16 configurations carrying nonzero amplitude. The counting register reads

outcome c	0	4	8	12	all others
probability	0.250000000	0.250000000	0.250000000	0.250000000	0

Table 2: Counting-register distribution, total 1.000000000000.

Continued fractions on $4/16$ and $12/16$ give $r = 4$; then $\gcd(7^2 - 1, 15) = 3$ and $\gcd(7^2 + 1, 15) = 5$.

The zeroes in Table 2 are the point. Twelve of the sixteen outcomes are operationally reachable — some derivation of the term produces each of them — and carry amplitude zero. By Definition 2.2 they constitute the interference deficit $\mathcal{D}(P)$, and by Corollary 6.6 the inclusion $\text{supp}(\llbracket P \rrbracket) \subseteq \text{Reach}(P)$ is strict by exactly those twelve. Shor’s algorithm, in this semantics, *is* a large interference deficit arranged on purpose.

9.6 The hazard: unnormalised clauses buy post-selection

§3.3 chose the unnormalised reading, with the Born rule applied at observation. That was the right call for the degenerate race, but nothing in §§5–6 requires a clause matrix to be an isometry. A non-isometric clause shrinks the norm, and renormalising at observation is precisely post-selection on the surviving branch.

Proposition 9.10 (Overshoot). *With $\mathbb{V} = \mathbb{C}$ and arbitrary clauses, the graded fragment with the Born reading decides PostBQP, and hence PP.*

Proof sketch. Lemmas 9.2–9.4 give universal quantum circuits. The clause $\text{diag}(1, \epsilon)$ is expressible and non-isometric; applying it and normalising at observation implements post-selection on the first outcome up to error $O(\epsilon)$. Conversely the model is simulable with post-selection. Aaronson’s identification of PostBQP with PP gives the second equality. $\square$

Measured, to show that the semantics charges nothing for it: a Hadamard layer on n qubits followed by $\text{diag}(1, 10^{-3})$ on each wire leaves a surviving norm of 2.5×10^{-1} , 1.25×10^{-1} , 6.25×10^{-2} , 3.13×10^{-2} for $n = 4, 6, 8, 10$ — falling as $2^{-n/2}$ — while the renormalised probability of the all-zero string is 0.999996, 0.999994, 0.999992, 0.999990. The filter is free.

So the fragment as specified in §6 is not a quantum programming language. It is a post-selected one, and it computes PP — far more than Shor, and far more than any device. Something must contain it.

The previous version of this note responded with a side condition, stated next. That response was insufficient, for the reason given in Remark 10.1: the deficiency it names is not

a property of badly-chosen clauses but of a semantics that defers normalisation, so it wants a construction rather than a filter. §10 supplies the construction; the condition below survives as the classification of the sites at which that construction is trivial.

Definition 9.11 (Isometric clause). Let r be a receipt site, let I index the input configurations on which r is enabled, and let O index the outcomes. The clause ψ at r is *isometric* when the matrix $A_{io} := \sum_{\sigma \in \text{Cand}: \text{out}(\sigma)=o} \llbracket \psi \rrbracket(\sigma)$, taken at input i , satisfies $A^\dagger A = \mathbf{1}$.

Proposition 9.12 (Decidability of the condition). *For a clause of the normal form of Theorem 7.2 with constant $v_i \in V$ and crisp χ_i , the matrix A is a finite table computable from the clause and the site’s arity, and isometry is decidable by inspection. For clauses using the graded modality $\langle K \rangle_{\mathbb{V}}$ the entries depend on the reachable successors and no such decision procedure is immediate.*

This gives a natural stratification, and it is the same one extraction wants: *the modality-free fragment with isometric clauses is the extractable fragment*, and it is the fragment a compiler to circuits can consume. Clauses using $\langle K \rangle_{\mathbb{V}}$ remain available for the stochastic and inference-driven regimes of §11, where no unitarity is expected of them.

In version 4 the stratification keeps that role and loses its other one. It no longer decides which programs are admissible — Corollary 10.11 shows it decides which heralds a compiler may erase.

9.7 What the example shows, and what it does not

It shows that expressiveness is not the obstacle: the fragment is universal for quantum circuits, and the classical portion of a quantum algorithm is written as ordinary rholang.

It does not show that graded rho has a quantum idiom of its own. Every gate above is a hand-built join gadget, and mobility, reflection and name-passing sit idle throughout; the encoding is circuit-shaped, and a circuit-shaped encoding demonstrates capability rather than fit. The one primitive the design offers that a circuit does not hand over directly is Theorem 8.2: an m -fold join is a BosonSampling instance in a single term former. Since candidate sets are permutation sets, the ambient symmetry group of the design is S_m rather than $\mathbb{Z}_N$, which suggests that a native algorithm — if there is one — would sit nearer the non-abelian hidden subgroup problem than near Shor. We record the direction without claiming it.

Finally, writing Shor is not running Shor. Evaluating the denotation classically is exponential; the simulator used for this section is a state-vector simulator wearing a process calculus. The value of the term is as a *compilation source*, and Proposition 9.12 says exactly which terms a compiler can accept.

10 Dilation, heralds, and the price of erasure

§9.6 left the fragment with a containment requirement and no mechanism. This section supplies the mechanism. It is not a restriction on which programs may be written; it is an amendment to the reduction rule that makes every step norm-preserving, together with an accounting rule that charges for the part of a program that would otherwise have been free.

10.1 The fork of §3.3 was not a trichotomy

The first version of this note offered three resolutions of the degenerate race and took the third. That presentation was wrong in a way worth stating plainly, because the correction is what drives everything below.

Remark 10.1 (Two options, not three). The unnormalised reading is not an alternative to the branch register; it *is* the branch register with the register traced out. Deferring normalisation to observation and then dividing by the surviving norm is the same operation as retaining the derivation index, discarding it, and conditioning on the survivors. Post-selection is the deferral of normalisation, and nothing else.

The consequence is that §9.6’s hazard and the absence of a per-step normalisation are one defect seen from two sides. Definition 9.11 responded to it with a side condition where a construction was available, and §6 already applies that construction once: Definition 6.4’s fictitious identity jump at normal forms exists precisely to complete a deficient row so that histories of different lengths are comparable. What follows does uniformly what the semantics already does in one special case.

10.2 The defect, dilated per site

Definition 10.2 (Defect operators). Let A be the clause matrix of a receipt site in the sense of Definition 9.11, and suppose $\|A\| \leq 1$. Its *defect operators* are

$$D_A := \sqrt{\mathbf{1} - A^\dagger A}, \quad D_{A^\dagger} := \sqrt{\mathbf{1} - AA^\dagger}.$$

Definition 10.3 (Heralded step). The *heralded step* at a site with clause matrix A is the map

$$V_A : |\gamma\rangle \mapsto A|\gamma\rangle \otimes |\mathbf{ok}\rangle + D_A|\gamma\rangle \otimes |\mathbf{fail}\rangle,$$

where the second factor is a fresh two-dimensional register attached to the site. Its unitary completion is the Julia–Halmos block

$$U_A = \begin{pmatrix} A & D_{A^\dagger} \\ D_A & -A^\dagger \end{pmatrix}.$$

The two-defect form is used rather than the familiar one-defect shorthand because a clause matrix arising from a directed rewrite need not be normal, and the shorthand is only unitary when it is.

Proposition 10.4 (The dilation is faithful). *For every contraction A , the map V_A is an isometry and U_A is unitary; and the $\mathbf{ok}$ block of U_A is A itself, unaltered.*

Proof. $V_A^\dagger V_A = A^\dagger A + D_A^\dagger D_A = \mathbf{1}$ by Definition 10.2; the block identities for $U_A^\dagger U_A$ are the standard ones and use $AD_A = D_{A^\dagger}A$. The second claim is immediate from the shape of the block, and is what makes the amendment conservative: on the branch where nothing was lost, the semantics of §6 is unchanged. $\square$

Computed over the filter clause $\text{diag}(1, 10^{-3})$ together with 200 pseudorandom contractions: the worst deviation of $U_A^\dagger U_A$ from the identity is 2.220×10^{-15} , and the worst deviation of the $\mathbf{ok}$ block from A is exactly zero.

Remark 10.5 (Contraction is the whole bill). Dilation requires $\|A\| \leq 1$: a clause may not manufacture weight. That is a scale bound, not a reversibility condition, and its $\mathbb{R}_{\geq 0}$ shadow is substochasticity, which the Gillespie reading wants in any case. Completion then turns the inequality into an equality. Nothing further is asked of a clause — in particular, purity of guard evaluation, which the existing condition evaluator already enforces, is all the reversibility the construction consumes.

Remark 10.6 (Per site, not per step). A companion note dilates the *global* one-step operator, adjoining one ancilla per step and recovering the n -step map by post-selecting all ancillas on zero. That construction is correct for a sequential machine and wrong here, for a reason internal

to this note: Proposition 4.2 says a global step is not the right unit, since the number of steps taken to reach a configuration depends on the interleaving and not on the causal history — the measured inflation is 2, 6 and 24 for two, three and four independent sites. An ancilla count indexed by global steps is therefore not an invariant of the derivation. Indexing instead by *site firing* makes the ancilla family coindexed with the derivations themselves, which is the same index the compilation route uses for its branch register. We record the difference rather than reconciling it here; the two notes disagree about what a step is, and this one is committed by Proposition 4.2.

Remark 10.7 (The fictitious jump, generalised). When A is diagonal in the configuration basis — the case of a clause that gates rather than mixes — D_A is diagonal too, and the **fail** branch is literally the unfired term carrying amplitude $\sqrt{1 - |a|^2}$. Definition 6.4’s fictitious identity jump is then exactly the **fail** branch of a site whose success amplitude is zero. One construction, two uses.

10.3 The herald

The register of Definition 10.3 is not a metatheoretic object. It is a message.

Definition 10.8 (Herald channel). For a receipt site r , its *herald channel* is $h(r) := @r$, the quotation of the site. Freshness is by quoting, so the address is deterministic, computable before execution, and identical across the candidates of the site.

Definition 10.9 (Heralded reduction). The communication rule is amended so that the outcome of a firing at r carries one additional message: $h(r)!(\text{ok})$ on the A branch, and $h(r)!(\text{fail})$ on the D_A branch, where the latter leaves the term component in the input sector. No term former is added, and no new sort of name: **ok** and **fail** are ordinary processes and $h(r)$ is an ordinary channel.

Two things now have to be checked, because a herald is a message and a message is part of a configuration. By Corollary 8.3 anything a continuation writes that distinguishes its inputs destroys the interference at that site. A herald is written by every site, so if it distinguished anything it would destroy everything.

Proposition 10.10 (The herald carries the defect bit only). *Let r be a site whose candidates satisfy the symmetry condition of Definition 8.1. If the herald emitted at r ranges over $\{\text{ok}, \text{fail}\}$, the outcomes of coinciding candidates remain congruent and Theorem 8.2 holds unchanged. If instead the herald names the candidate, the outcomes separate, every fibre of out becomes a singleton, and the coherent sum degenerates to the incoherent one at every site of the program.*

Proof. Immediate from Corollary 8.3: coincidence of outcomes is congruence of the resulting terms, and appending the same message to congruent terms preserves congruence, while appending distinct messages destroys it. $\square$

So the herald is the branch register *quotiented* onto success — the coarsening that forgets which candidate fired and remembers only whether the step preserved norm. Measured on the minimal interferometer of §8.2, with relative phases 0, π and $\pi/2$: the bare coherent weights are 2.000000, 0.000000, 1.000000; with an **ok** herald appended they are 2.000000, 0.000000, 1.000000; with a candidate-naming herald they are 1.000000, 1.000000, 1.000000. The cancellation survives the first and does not survive the second.

Corollary 10.11 (Uniform heralding is free where it should be). *At a site whose clause is isometric in the sense of Definition 9.11, the fail branch carries amplitude zero, so the herald reads **ok** on every candidate and is inert. Hence heralding every receipt costs nothing exactly on the fragment Proposition 9.12 can identify.*

This is why the amendment can be uniform rather than selective. One does not have to decide in advance which sites are deficient: the deficient ones herald informatively and the rest herald constantly, and Proposition 9.12 is demoted from an admission criterion to an optimisation, telling a compiler which heralds may be elided.

Remark 10.12 (Uncompute, or read). A herald that is never consumed sits in the term as a record. At an isometric site the record is constant and therefore not a record at all. At a deficient site it is a genuine one, and leaving it unread is a decoherence event — which is the correct semantics for a program that discards information, not a malformedness to be refused. Reading it is the retry. This is the operational content of the requirement that a dilation ancilla be uncomputed rather than traced or projected: here it is neither traced nor projected but *addressed*.

10.4 Post-selection becomes a program

With Definition 10.9 in force, a programmer who wants the behaviour that Proposition 9.10 obtained for free must write it:

```
new done in {
  contract attempt(_) = {
    prepare!(Nil)
  |
    for( s <- h(r) ) {
      match *s {
        ok    => { done!(Nil) }
        fail  => { attempt!(Nil) }
      }
    }
  } | attempt!(Nil)
}
```

Proposition 10.13 (Post-selection is not a step). *Under Definition 10.9 no step of the semantics loses norm. Consequently the renormalisation performed at observation in §6.3 has nothing to do, and the operation $\text{diag}(1, \epsilon)$ of Proposition 9.10 is no longer a step: it is a heralded step followed by a projection onto ok , and projection is not among the constructs of the semantics. It is available only as the retry loop above, whose failures are firings.*

Definition 9.11 accordingly loses its status as a containment requirement and becomes what it always described: a classification of sites, the isometric ones being those at which the dilation is trivial. Nothing is refused.

10.5 The price

The retry loop is a program, and programs are funded. The resource discipline already in place admits a payment exactly when the balance covers it, and a loop that does not terminate within its endowment simply stops.

Proposition 10.14 (Expected attempts). *Let a site herald ok with probability p , and let the retry loop restore the site's inputs before each attempt, so that attempts are independent. Then the expected number of firings before an ok is $1/p$, and the expected charge is κ/p where κ is the cost of one firing.*

Applied to the hazard of §9.6, whose figures were computed for that section and are here re-read as a bill:

n	surviving norm	$\Pr[\text{all heralds ok}]$	expected attempts
4	0.250001	0.062500	16
6	0.125000	0.015625	64
8	0.062500	0.003906	256
10	0.031250	0.000977	1024

The filter that was free in §9.6 costs 2^n firings. This is the whole of the containment argument, and it is worth stating as such.

Conjecture 10.15 (Containment by the ledger). *A graded program whose execution is funded by a budget polynomial in the size of the term does not decide PP. The post-selection witnessing Proposition 9.10 requires a success probability exponentially small in the number of wires, hence by Proposition 10.14 an exponentially large expected charge, which no polynomial endowment funds.*

Remark 10.16 (Where the conjecture is soft). Proposition 10.14 assumes attempts are independent. A retry loop whose success probability depends on what earlier attempts returned is not geometric, and a sufficiently clever adaptive schedule is exactly the resource that makes linear optics universal. So Conjecture 10.15 and Obligation 14.6 are the same question asked with opposite hopes: one wants adaptivity to be cheap enough for universality and expensive enough for containment, and the two demands meet somewhere that has not been located.

10.6 Why not the linear fragment

There is an alternative to paying at the herald, and it should be recorded because it is the first thing a reader who knows linear logic will propose: restrict the calculus so that the defect cannot arise. Weakening — a bound name not used in the continuation — is the partial trace, and contraction is the copy. A discipline in which every name bound by a for-comprehension is used exactly once aligns the term language’s structural rules with those of the target, and makes the isometry of a step structural rather than conditional.

The reason not to take it is that the expressiveness of §9 is bought with weakening, and precisely with weakening.

Proposition 10.17 (Weakening-free steps are block-diagonal). *Suppose every name bound at a site is used at least once in the continuation, so that the payload multiset of the outcome is the consumed multiset together with whatever the continuation writes literally. Then outcomes arising from distinct input configurations are never congruent, the one-step operator is block-diagonal with respect to the input basis, and no site realises a basis-changing map.*

Proof sketch. Distinct input basis states present distinct payload multisets at the site’s channels. If nothing is discarded, the outcome’s payload multiset determines the input’s, so no outcome lies in the image of two distinct inputs. The fibres of out over distinct inputs are therefore disjoint, and disjoint supports give a block-diagonal matrix. $\square$

The gate gadget of Lemma 9.2 discards both the input datum and the unselected ancilla, and Remark 9.3’s second erasure — the coincidence of outcomes across distinct *inputs*, as opposed to across distinct candidates — is exactly what the discard buys. Patching the gadget by re-emitting what it discarded does restore the isometry, structurally and with no side condition; it restores it by making the map block-diagonal. Computed on the patched gadget with a clause that was to have been a Hadamard: the two rows have supports $\{0, 1\}$ and $\{2, 3\}$, the intersection is empty, and $\|MM^\dagger - \mathbf{1}\| = 2.220 \times 10^{-16}$. An isometry, and not a gate.

Remark 10.18 (The two erasures come apart). The restriction does not damage §8. The continuations there — $z!(*y_1) \mid z!(*y_2)$ and its asymmetric sibling — already use each bound name exactly once, so Theorem 8.2 is a theorem of the linear fragment. Erasure across candidates is achieved by symmetry and is linear; erasure across inputs is achieved by discard and is not. Only the second is implicated.

Remark 10.19 (The road not taken). Proposition 10.17 is a statement about the encoding of Definition 9.1, in which a qubit is *which datum* sits on a channel. Encode instead by *which channel* a fixed datum sits on, and mixing becomes redistribution, which is what a linear join does; the payload multiset is then conserved by construction and the model is linear optics rather than circuits. Reflection makes this natural rather than laborious, since a received name may be used as the channel of a send and so the output rail can be selected by the candidate. That route buys unitarity by construction and pays for universality at Obligation 14.6 instead. It is a different note, and this one keeps the circuits.

Principle 10.20 (Where the bill falls). A design may have structural unitarity or circuit-shaped expressiveness, and the choice determines where the price of erasure is paid: at the encoding, or at the herald. Paying at the herald leaves the language alone and makes the payment visible in the ledger.

11 PLN-valued where-clauses

11.1 Why this slot invites PLN

MeTTaIL describes finitely presentable graph-structured lambda theories, and the control-flow layer is a rholang whose channels are typed by those language definitions. Probabilistic Logic Networks are the uncertain-inference framework developed for the OpenCog and Hyperon line of work, in which knowledge is annotated with truth values and inference rules carry truth-value formulas computing a conclusion’s annotation from its premises’.

The proposal of this note opens a door between them. If a **where** clause is valued in an algebra rather than in the Booleans, then a clause may be valued *by an inference*: the propensity with which a communication fires is the truth value PLN currently assigns to the proposition that it should. Execution stops being gated by belief and starts being *driven* by it. The scheduler and the reasoner become the same loop.

This is a different use of the machinery from the quantum one, and it is worth being explicit that it is a different *regime* rather than a different mechanism.

11.2 PLN truth values, briefly

We use PLN’s *simple truth values*: a pair $(s, c) \in [0, 1]^2$ of a strength, the estimated probability that a statement holds, and a confidence, reflecting how much evidence stands behind that estimate.¹ The independence-based deduction formula for term-logic transitivity is

$$s_{AC} = s_{AB}s_{BC} + \frac{(1 - s_{AB})(s_C - s_{BSBC})}{1 - s_B},$$

with a concept-geometry variant replacing the independence assumption by an assumption about the shape of concepts, $s_{AC} = s_{AB}s_{BC} / \min(1, s_{AB} + s_{BC})$. Induction and abduction are obtained by chaining deduction with Bayes’ rule. The quantity $\text{pow} := s \times c$, sometimes called the power of an uncertain statement, behaves multiplicatively under deduction in the regime where term probabilities are highly uncertain: $\text{pow}_{AC} \approx \text{pow}_{AB} \text{pow}_{BC}$.

¹Confidence is formalised in several ways across the literature, including as the width of a confidence interval around the mean of a second-order distribution; the NARS-style count form $c = n/(n + k)$ for an evidence count n and a personality parameter k is the one easiest to read operationally, and we use it informally.

11.3 The resolution algebra, honestly

The design of §5 requires a commutative semiring. PLN’s operations do not straightforwardly furnish one, and it is better to say where the difficulty lies than to paper over it.

Multiplication is unproblematic. Conjunction of independent statements multiplies strengths, and confidences compose multiplicatively under the usual independence assumption, so $\otimes$ may be taken componentwise; under the high-uncertainty regime this agrees with deduction on the power.

Addition is the problem. PLN’s operation for combining two estimates of the same proposition is *revision*, a confidence-weighted average

$$s = \frac{s_1 c_1 + s_2 c_2}{c_1 + c_2},$$

with an accompanying rule for the revised confidence. Revision is not the addition of a semiring: it is idempotent on equal arguments, it does not have $(0, \cdot)$ as a two-sided unit in the required sense, and it does not distribute over componentwise multiplication. Taking $\oplus$ to be revision therefore makes the one-step operator (2) sensitive to the order in which candidates are aggregated, which is precisely the order the calculus refuses to fix.

Two responses are available, and we recommend the first.

1. **PLN supplies values; $\mathbb{R}_{\geq 0}$ supplies the algebra.** Let the graded sort contain ground formulae whose evaluation is a PLN inference, and let the clause’s value be the resulting power $\text{pow} = s \times c \in [0, 1]$, with $\oplus = +$ and $\otimes = \times$ over $\mathbb{R}_{\geq 0}$. Then $\mathbb{V} = \mathbb{R}_{\geq 0}$, the semiring laws hold, and the whole apparatus of §§5–6 applies unchanged. Deduction’s multiplicativity on the power means that chaining inference and chaining communication agree, which is the coherence one wants. The confidence coordinate is not discarded: it enters the propensity, so a poorly-evidenced belief yields a rarely-taken branch rather than a forbidden one.
2. **Carry (s, c) and weaken the algebra.** Keep the pair as the carrier and accept a lax structure in which aggregation is revision. This is more faithful to PLN but requires a canonical aggregation order, which must then be justified as not observable — a substantial obligation, and one that reintroduces exactly the kind of implementation-visible choice the design was meant to abstract.

Principle 11.1 (Fuzzy inference drives execution). Under the first response, a MeTTaIL rewrite fires with propensity equal to the power PLN currently assigns to the proposition, expressed as a graded ground formula in the rule’s **where** clause, that the rewrite is warranted here. Because clauses are re-evaluated at every candidate against the current term (Proposition 7.4), revisions of belief take effect on the next step without any separate update mechanism.

11.4 Three regimes, one syntax

The point of Table 1 is now visible. The same slot, with the same formula language and the same per-match evaluation, gives:

- $\mathbb{B}$ — rholang 1.4, unchanged, by Theorem 6.5;
- $\mathbb{R}_{\geq 0}$ — stochastic execution, with PLN as a distinguished source of values, in which inference schedules computation;
- $\mathbb{C}$ — coherent execution with interference, in which the clause is a scattering matrix by Theorem 8.2.

It is worth stating the boundary between the second and third plainly, since it is easy to blur. PLN values live in $[0, 1]$ and carry no phase; PLN-driven execution is therefore stochastic and exhibits no cancellation. The interference regime requires additive inverses in the value algebra, which is a property of $\mathbb{C}$ and not of any probability-valued logic. What the two share is the mechanism; what separates them is whether $\oplus$ can cancel.

Remark 11.2 (A speculation worth naming, and not indulging). PLN has been given paraconsistent formulations, and paraconsistent logics are one of the places complex or otherwise non-order-theoretic truth values have been contemplated. Whether a paraconsistent PLN would furnish a value algebra with cancellation — and hence let inference and interference share not merely a slot but a semantics — is a real question and not one this note attempts.

12 A second worked example: a mortal forager

§9 exercised the coherent regime. This section exercises the inference-driven one, and it is drawn from a setting where graded clauses have somewhere to go rather than being demonstrated in the abstract: the mortal scientist programme, in which a learner’s hypotheses are already installed in **where** guards and every observation is charged against a finite token stack. There, a hypothesis in operational position is an experiment. Here we value that guard in PLN and ask what changes.

The example is deliberately small — one forager, one predicate, one belief — and every number below is measured by a simulator that runs the clause as specified.

12.1 The setting

A forager meets a stream of specimens. Each is *rich*, carrying a stack of $\sigma_R = 120$ tokens, or a *decoy*, carrying $\sigma_D = 0$. Breaking one open costs $\kappa_{\text{break}} = 40$, so opening a decoy is a pure loss. A structural predicate χ at depth two costs $\kappa_{\text{sense}} = 2^2 = 4$ to evaluate and is *evidence*, *not a decision procedure*: it is satisfied by 80% of rich specimens and 30% of decoys. The forager may instead graze a wall yielding 5, which is the outside option. It starts with 120 tokens and meets at most 400 specimens; death is failure to fund.

Two worlds. In world *A* the base rate of rich specimens is 0.40, so $\text{Pr}[\text{rich} \mid \chi] = 0.64$ and opening a χ -specimen is worth +36.80 in expectation. In world *B* the base rate is 0.15 and χ is satisfied by 40% of decoys, so $\text{Pr}[\text{rich} \mid \chi] = 0.26$ and opening is worth −8.70. The forager is not told which world it is in.

```
new bel, prey, wall in {
  bel!( (1, 0.5) ) |                                     // (n, s): one unit of prior evidence

  // the two branches race for the belief and the specimen
  for( @(n,s) <- bel ; @spec <- prey where Chi(spec) * Pow(n,s) ) {
    Break!(spec) | bel!( revise(n, s, spec) )
  } |
  for( @(n,s) <- bel ; @spec <- prey where g ) {
    Graze!(*wall) | bel!( (n, s) )
  }
}
```

Chi is a crisp structural predicate entering through the indicator embedding $[-]$; $\text{Pow}(n, s)$ is a graded ground formula evaluating to the PLN power $s \cdot c$ with $c = n/(n + k)$; $*$ is the graded conjunction $\otimes$; and g is a scalar constant of $\mathbb{V} = \mathbb{R}_{\geq 0}$. By §5.4 both clauses are evaluated per candidate, so the forager’s propensity to open *this* specimen is sc against the wall’s g .

Remark 12.1 (The belief is a message). The belief sits on a channel as data. It is therefore in the tuple space like any other datum, which means it can be read, raced for, and taken. In the ecological reading this is not an implementation detail: testimony and the theft of a hypothesis are the same operation as predation, differing only in which namespace the name lives in.

12.2 What the graded clause replaces

The assay in the mortal scientist programme is a *complementary guard pair*: one receipt guarded by φ reporting confirmation, a second guarded by $\neg\varphi$ reporting refutation, and a third outcome $\perp$ when neither fires before the budget is exhausted. The pair is needed because a Boolean guard that does not fire is indistinguishable from one still waiting.

This has a dependency worth naming: the second guard requires $\neg$ *in the hypothesis language*, and a generated logic supplies negation only when the target logic specification does — which several members of the generating family do not.

Proposition 12.2 (The trichotomy is the corners of the truth-value square). *Valuing the guard at a PLN simple truth value (s, c) replaces the pair by one clause and its complement in the value algebra: the confirming branch carries sc , the refuting branch $(1-s)c$, and neither fires with weight $1-c$. Confirmation, refutation and $\perp$ are then the corners $c = 1, s = 1$; $c = 1, s = 0$; and $c = 0$ of the unit square, and $\perp$ becomes a coordinate rather than a residue.*

The negation the assay needs has moved from the formula language, where it may not exist, to $[0, 1]$, where $1-s$ always does. This is the same observation as §5’s — that negation does not survive to the graded sort — read in the direction where it is a gain rather than a cost.

Remark 12.3 (Confidence needs an outside option to be visible). If the competing branch is the complement of the same belief, the confidence coordinate is a common factor: $sc/(sc+(1-s)c) = s$ exactly, and c cancels. Confidence changes behaviour only against an alternative whose weight does not scale with it. In this example that alternative is the wall, and in the ecology generally it is autotrophy. Grading a guard buys nothing unless the learner has something else to do.

12.3 The bootstrap trap

Proposition 12.4 (Zero is a fixed point of the power). *A belief with no evidence has $c = 0$, hence power 0, hence a clause value of 0, hence — by the support reading of Remark 5.3 — a receipt that never fires. It therefore never acquires evidence.*

Measured: a forager initialised with $n = 0$ opens 0.00 specimens over 400 encounters in either world and ends with 520.0 tokens, exactly the grazing baseline, against 3543.7 for the same forager given a single unit of prior evidence in world A . The trap is not death but stasis.

The remedy is available inside the clause language and is worth stating as such: either the belief carries prior evidence $n_0 \geq 1$, which is PLN’s own convention, or the clause is $\text{Pow}(n, s) \oplus \varepsilon$ for a scalar ε — *exploration is a disjunct*. We use the former below.

12.4 Two worlds, four foragers

Four arms. NAIVE has no prior evidence. CRISP is the mechanism as it stands: the guard is χ , weight 1 when satisfied, confidence invisible. INDIVIDUAL holds a hypothesis about the specimen, so its evidence count cannot pass 1 — the specimen is destroyed by the assay. PLN holds a class hypothesis and accumulates. All at $k = 4$, over 20,000 lives each.

Two things in this table are worth stating as findings rather than data.

First, in world A the crisp forager has the *highest* mean terminal stack and the *lowest* survival. The confidence coordinate does not buy yield; it buys variance. A forager that acts

world	arm	survival	terminal	lifetime	opened	decoys
<i>A</i>	NAIVE	1.0000	520.0	400.0	0.00	0.00
<i>A</i>	CRISP	0.8501	4084.7	341.6	113.83	40.94
<i>A</i>	INDIVIDUAL	0.9478	2117.3	381.2	50.68	18.20
<i>A</i>	PLN	0.9485	3543.7	381.6	95.53	34.36
<i>B</i>	NAIVE	1.0000	520.0	400.0	0.00	0.00
<i>B</i>	CRISP	0.0029	27.4	30.5	8.72	6.44
<i>B</i>	INDIVIDUAL	0.2121	95.2	166.8	13.65	10.10
<i>B</i>	PLN	0.0509	37.2	125.2	14.85	10.99

Table 3: Four foragers, two worlds, $k = 4$.

on a satisfied guard from its first encounter does better on average and dies more often, which is the trade a mortal learner is actually facing and which a Boolean guard cannot express.

Second, and more sharply: in world *B* the crisp forager is essentially extinct at 0.29% survival. It cannot do otherwise. Its policy is the formula, and the formula does not change, so a *Boolean guard has no coordinate in which to record disappointment*. The graded forager’s policy is the formula’s *value*, which moves. Here is a single life in world *B*:

opening	s	c	power	Pr[open]
1	0.7500	0.3333	0.2500	0.3333
2	0.5000	0.4286	0.2143	0.3000
3	0.3750	0.5000	0.1875	0.2727
5	0.2500	0.6000	0.1500	0.2308

Table 4: Learning to stop. Strength falls as decoys accumulate; confidence rises; the product falls, and with it the propensity to open.

Remark 12.5 (Exploration and exploitation are not scheduled). Nothing in the clause is a schedule. Early behaviour is cautious because c is small, late behaviour is decisive because c is large, and the transition rate is set by how fast evidence arrives. The usual hyperparameter is absent because the quantity it approximates is a coordinate of the truth value.

12.5 Amortisation, measured

The class hypothesis and the individual hypothesis differ in exactly one place: the individual’s evidence count cannot grow, because the assay destroys its subject. That is not a difference in what can be believed but in what can be *accumulated*, and PLN’s confidence is the ledger of accumulation.

Measured over survivors in world *A* at $k = 4$, against the true posterior 0.6400:

arm	mean $ s - \text{truth} $	mean evidence count	survivors
INDIVIDUAL	0.2039	2.00	18,917
PLN	0.0383	101.54	18,900

A factor of 5.3 in accuracy, for evidence that costs nothing extra — the openings happen either way; only the bookkeeping differs. This is the amortisation advantage in its defensible form. It says that reusable class hypotheses recover their acquisition cost across instances and individual ones cannot, which is a selective advantage; it does not say that individual hypotheses have no value, and the table shows they do not (the INDIVIDUAL arm outlives CRISP in both worlds).

12.6 The personality parameter acquires a metabolic optimum

In PLN and its relatives, k is a free parameter: the amount of evidence at which confidence reaches one half. It is chosen by the designer’s taste. In a mortal ecology it is not free, because it is priced on both sides. A credulous forager (small k) becomes confident before it has grounds and opens decoys it cannot afford; a sceptical one (large k) never becomes confident enough to eat.

Scored by mean log terminal wealth — the criterion appropriate to a multiplicative, absorbing process, where ruin contributes zero rather than a small number — the optimum is interior. World A , 40,000 lives per point:

k	4	5	6	7	8	9	10	12	14	16	20
survival	.944	.952	.959	.967	.970	.976	.979	.984	.988	.991	.995
log-fitness	7.744	7.785	7.818	7.853	7.852	7.874	7.872	7.858	7.832	7.798	7.715

Table 5: An optimal degree of scepticism, $k^* = 9$.

In an ensemble where each life draws world A or world B with equal probability — the honest setting, since a learner does not know which world it is in — the optimum moves to $k^* = 64$, with log-fitness 6.343, **6.431**, 6.354, 6.316, 6.263 at $k = 32, 64, 96, 128, 512$. In world B alone it runs to the boundary, because when the evidence is a trap the best policy is abstinence and large k is how the forager approximates it.

Principle 12.6 (Scepticism is priced by the environment). k^* is not a property of the reasoner. It rises with the risk of the world the reasoner is in — here by a factor of about seven between a world whose evidence is sound and an ensemble containing a trap. A parameter that uncertain inference leaves to taste is, in a mortal setting, selected.

We regard this as the example’s main return, and note that it runs in both directions: the graded clause gives the ecology a principled exploration schedule, and the ecology gives PLN a selection criterion for a parameter it otherwise has to stipulate.

12.7 Up the fractal

The forager is a single learner. The programme this example is drawn from continues upward — to learners that compose by parallel composition over shared namespaces, to lineages, and to populations — and the graded clause has something to say at each level. We record two observations and claim neither.

The first is that PLN’s operation for combining two estimates of the same proposition is *revision*, which adds evidence counts. Two learners that share a hypothesis namespace and exchange truth values therefore have superadditive confidence, in a setting where the same programme’s own results make metabolism subadditive: tokens are conserved and formulae are not. Testimony would then be the operation at which a composite strictly exceeds the sum of its parts, and PLN would say by how much.

The second is the hazard attending the first. Revision assumes the evidence being combined is independent. Two learners who both heard from a third double-count, and confidence rises with no evidence behind it. *Belief inflation is the epistemic analogue of minting* — and where the conservation law of a token economy forbids minting outright, nothing in the epistemic layer forbids this. Whether a namespace discipline can restore conservation to the evidence ledger seems to us the right question to ask next.

Both of these need the truth value carried as a pair rather than collapsed to its power, which puts them outside the algebra recommended in §11 and into the lax structure set aside

there. They belong to a subsequent paper, along with the back-patching of the results above into the programme they came from.

13 Implementation

The design is deliberately small on the implementation side, and most of what it needs already exists.

Candidate enumeration. The store presentation gives the redex enumerator directly: the map from channels to rows of data and waiting continuations, together with the join index, is precisely Cand. The one requirement the current representation does not meet is occurrence identity — per-channel occupancy counts collapse the distinction between congruent data, and identifiers allocated during firing are not stable across branches. Candidates must be addressed by (channel, canonical slot), deterministically and branch-independently. This is the same addressing obligation that arises in the compilation route, and it should be solved once.

Basis identity. Interference is addition of coefficients on a common basis vector, so the implementation needs a canonical name for a term modulo structural congruence. The content-addressed store already supplies one. This also repairs a known defect: a configuration fingerprint that formats values to fixed decimal precision cannot distinguish 1, -1 and i , and therefore cannot represent the very distinctions the grading exists to make.

Clause evaluation. The crisp sort is the existing condition language and its evaluator; the graded sort adds scalars, two binary operations and the graded modality. The modality is the only construct requiring successor exploration, and it takes the inference budget.

Two readings out. A simulator should emit both the support/Born reading and the coherent sum, since by Corollary 6.6 they differ exactly on the interference deficit. If they never differ on a corpus of test theories, the grading is doing nothing and the implementation is wrong.

Heralds. The amendment of Definition 10.9 touches the communication rule and nothing else: on firing, emit one further message on the quotation of the site. Two consequences for an implementation. The herald’s payload must be the defect bit, never the candidate index (Proposition 10.10); and heralds at isometric sites should be elided at elaboration, which is the residual job of Proposition 9.12. Unelided heralds accumulate as messages, and a garbage discipline for them is a real cost that this note does not price.

What retires. Partition construction and its checked funnel; the requirement that keys be structural, as a precondition for partition locality; the update-function plumbing and the declarative update sub-language it wanted; and the class-global phase, which was in any case observationally inert.

14 Open threads

Obligation 14.1 (Length-asymmetric reconvergence). Theorem 8.2 secures interference at a single join. The general question is which closed loops survive the causal quotient of Definition 4.1 in pure rho — that is, whether two causally inequivalent derivations of different lengths can reach a common term. Reflection is the natural suspect, since dropping a quoted process

makes the residual communication count depend on which datum won. A negative answer would confine interference to joins; a positive one would open a second, non-local source.

Obligation 14.2 (Normalisation). *Partly discharged in version 4.* Remark 10.1 settles that the unnormalised reading is the branch register traced out, so the fork of §3.3 has two branches and not three, and §10 takes the register and keeps it addressable. What remains open is the observable question that motivated the original choice: whether the m -fold degeneracy of a race should be read as weight m or m^2 is now a question about which herald a program reads, and that reading should be worked out against Remark 3.2.

Obligation 14.3 (Enforcing isometry — withdrawn). *Withdrawn in version 4 and replaced by Obligation 14.4.* The previous version asked for a static discipline refusing non-isometric clauses in a declared coherent mode. §10 refuses nothing: the defect is dilated rather than excluded, so a non-isometric clause is a legal program with a herald worth reading. Definition 9.11 and Proposition 9.12 survive as the classification telling a compiler which heralds are constant.

Obligation 14.4 (The price, made exact). Conjecture 10.15 claims that a polynomially funded graded program does not decide PP. Two gaps. Proposition 10.14 assumes independent attempts, and Remark 10.16 notes that an adaptive retry schedule is not geometric — which is the same resource Obligation 14.6 needs for universality, so the two must be settled together. And the charge for a herald has not been fixed against the existing cost model; until it is, ‘polynomially funded’ names an intention rather than a judgement.

Obligation 14.5 (Herald garbage). Heralds at deficient sites accumulate as messages. Left unread they decohere their sites, which is semantically correct and operationally a leak. Whether a discipline exists that collects them without either reading them — which commits to an outcome — or discarding them — which is the trace this note set out to avoid — is open, and is the point at which Remark 10.12’s ‘neither traced nor projected but addressed’ stops being free.

Obligation 14.6 (Adaptivity and universality). A continuation may install further joins depending on what it received, which is adaptivity. Whether adaptive graded joins reach a universal fragment — the question KLM answers affirmatively for linear optics — is the question on which a verdict about advantage ultimately rests. Version 4 sharpens it in two directions. Remark 10.16 makes it the same question as Conjecture 10.15, asked with the opposite hope. And Remark 10.19 identifies the setting in which it would be asked natively: under a mode encoding, the linear fragment is linear optics, mixing is a join, and heralded measurement is already available as a herald read — so a mode-encoded graded rho would be the natural home of a KLM-style construction, and this note’s circuit encoding is not.

Obligation 14.7 (Categorical status). The construction looks like a Kleisli category for the free-module monad of $\mathbb{V}$ over terms modulo congruence, with the graded modality as the structure map of a coalgebra. Making this precise would connect the design to the existing coalgebraic account of the store denotation and its two-layer full abstraction.

Obligation 14.8 (Testimony, and conservation of evidence). §12.7. Carrying (s, c) rather than the power puts revision back in play, which makes confidence superadditive under testimony and opens the double-counting failure — belief inflation as the analogue of minting. Both require the lax algebra set aside in §11 and belong to a subsequent paper, together with back-patching §12’s results into the mortal-scientist programme they are drawn from.

Obligation 14.9 (Paraconsistent values). Remark 11.2: whether an inference calculus can supply a value algebra with cancellation.

15 Conclusion

The rho calculus does not say how its races are settled, and that silence is a parameter rather than a gap. Filling it differently gives different executions of the same program, and a programmer who wants to arrange interference needs to be able to write the filling down.

The change proposed here is one line of grammar: let the **where** clause of the for-comprehension take values in a commutative semiring, and evaluate it once per candidate match rather than once per communication. Boolean values give back today’s language exactly. Non-negative reals give stochastic execution, with probabilistic-logic inference as a natural source of values, so that what the system believes decides what it does next. Complex values give interference, and the clause becomes a scattering matrix whose permanent is the amplitude of the symmetric outcome.

Shor’s algorithm fits, and fitting it was informative in an unexpected direction. Universality is easy, the classical portion of the algorithm is ordinary rholang, and the algorithm’s characteristic behaviour shows up in this semantics as a large interference deficit arranged on purpose — twelve of sixteen reachable outcomes carrying amplitude zero. What the exercise exposed is that the fragment as first specified reaches too far: with clause matrices unconstrained, the renormalisation performed at observation is post-selection, and the fragment computes **PP**.

Version 4 closes that by amending the reduction rule rather than the admissible programs. The deferral of normalisation was the whole defect — the unnormalised reading is the branch register with the register traced out — so the register is kept, and kept where a rho program can reach it: as an ordinary message on a channel named by quoting the site, carrying one bit that says whether the step preserved norm. Three things fall out. The bit is the only thing the message may carry, since a herald naming the candidate would write a which-path record at every site and collapse the coherent sum everywhere. Because of that, heralding is inert precisely where isometry holds, so the amendment can be applied uniformly and the isometry condition, no longer needed to refuse anything, becomes the classification telling a compiler which heralds to erase. And post-selection stops being an operation of the semantics: it is a retry loop that a programmer writes and a ledger charges, which turns the free filter of the hazard into a bill of 2^n firings.

The obvious alternative was tried and rejected, and the rejection is worth keeping. Restricting to the fragment in which every bound name is used exactly once does make each step an isometry structurally, with no dilation and no herald. It also makes each step block-diagonal in the input basis, because the coincidence of outcomes across distinct inputs that a gate needs is bought precisely by discarding the input. The recombiner is untouched — its continuations were already linear — so the two erasures of the design come apart, and only the one that circuits depend on is implicated. A design may have structural unitarity or circuit-shaped expressiveness; what the choice fixes is whether the price of erasure is paid at the encoding or at the herald. Paying it at the herald leaves the language alone and puts the payment where it can be read.

The inference-driven regime was exercised too, on a mortal forager whose hypothesis is a PLN truth value sitting in a guard. Three things came out of it. The complementary guard pair such a learner needs in order to tell refutation from patience turns out to require negation in the hypothesis language, which a generated logic need not supply — and grading moves that negation into $[0, 1]$, where it always exists. A Boolean guard has no coordinate in which to record disappointment, so it cannot learn to stop, and in a world whose evidence is a trap it does not survive. And PLN’s personality parameter, which uncertain inference leaves to the designer’s taste, acquires an interior optimum that rises with the risk of the environment: scepticism, priced.

Two things about this were not evident at the outset. The first is that weight maps were not wrong but merely linear: they are the normal forms $\bigoplus_i v_i \otimes [\chi_i]$, and the partition discipline

that attended them was the price of making a lookup single-valued rather than a fact about weights. The second is that the recombination interference requires — the half of the problem no weight map addressed — was already in the language. A join over a contended channel has one candidate per assignment and one outcome per orbit of the continuation's symmetry, so whether a race interferes is decided by whether the continuation troubles to tell its inputs apart. That is a dial a programmer already knows how to turn.